
Analisis Pengaruh Harga retail dan Tipe Outlet Terhadap Pendapatan Penjualan Ritel



INTRODUCTION

Self-Overview

**Raden Maulana
Akbar**

- Education
 - SMAN 7 Bekasi
- Project
 - Employee Attrition & High-Performance Loss Analysis
 - Analisis Jenis Produk dan Visibilitas Ruang Pajang Terhadap Penjualan Ritel



Problem Statement



Analisis Jenis Produk dan Visibilitas Ruang Pajang Terhadap Penjualan Ritel

Dalam industri ritel modern, ketimpangan performa penjualan antar gerai sering kali menjadi kendala utama dalam optimalisasi pendapatan. Manajemen kerap menghadapi kesulitan dalam menentukan apakah fokus strategis harus diarahkan pada penyesuaian harga produk atau ekspansi format tipe gerai. Penelitian dan analisis ini berfokus untuk mengidentifikasi sejauh mana pengaruh harga eceran dan klasifikasi tipe outlet terhadap fluktuasi pendapatan penjualan, guna memberikan rekomendasi alokasi strategi bisnis yang tepat sasaran."

Data Exploration



Sumber: BigMart Sales Data.csv

Ukuran: 8523 baris × 12 kolom

Isi dari Raw Data

- Data Outlet (Outlet_Identifier, Outlet_Establishment_Year, Outlet_Size, Outlet_Location_Type, Outlet_Type, Item_Outlet_Sales)
- Data Produk (Item_Identifier, Item_Weight, Item_Fat_Content, Item_Visibility, Item_Type, Item_MRP)

```
df = pd.read_csv('BigMart Sales Data.csv')
df
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type	Outlet_Type	Item_Outlet_Sales
0	FDA15	9.300	Low Fat	0.016047	Dairy	249.8092	OUT049	1999	Medium	Tier 1	Supermarket Type1	3735.1380
1	DRC01	5.920	Regular	0.019278	Soft Drinks	48.2692	OUT018	2009	Medium	Tier 3	Supermarket Type2	443.4228
2	FDN15	17.500	Low Fat	0.016760	Meat	141.6180	OUT049	1999	Medium	Tier 1	Supermarket Type1	2097.2700
3	FDX07	19.200	Regular	0.000000	Fruits and Vegetables	182.0950	OUT010	1998	NaN	Tier 3	Grocery Store	732.3800
4	NCD19	8.930	Low Fat	0.000000	Household	53.8614	OUT013	1987	High	Tier 3	Supermarket Type1	994.7052

Data Exploration

```
df.info()
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 8523 entries, 0 to 8522  
Data columns (total 12 columns):  
#   Column                                Non-Null Count  Dtype  
---  ---                                -  
0   Item_Identifier                       8523 non-null   str  
1   Item_Weight                          7060 non-null   float64  
2   Item_Fat_Content                     8523 non-null   str  
3   Item_Visibility                      8523 non-null   float64  
4   Item_Type                            8523 non-null   str  
5   Item_MRP                             8523 non-null   float64  
6   Outlet_Identifier                    8523 non-null   str  
7   Outlet_Establishment_Year            8523 non-null   int64  
8   Outlet_Size                          6113 non-null   str  
9   Outlet_Location_Type                 8523 non-null   str  
10  Outlet_Type                          8523 non-null   str  
11  Item_Outlet_Sales                    8523 non-null   float64  
dtypes: float64(4), int64(1), str(7)  
memory usage: 1.2 MB
```

Hasil Pengamatan

Kolom Item_Identifier, Item_Fat_Content, Item_Type, Outlet_Identifier, Outlet_Size, Outlet_Location_Type, dan Outlet Type bertipe data jenis kategorik

Kolom Item_Weight, Item_Visibility, Item_MRP, Outlet_Establishment_Year, dan Item_Outlet_Sales bertipe data numerik

Kolom Item_Weight dan Outlet_Size memiliki jumlah data yang lebih sedikit dibandingkan kolom lainnya (terdapat missing value)

Kolom Outlet_Establishment_Year bertipe data integer belum bertipe datetime

Data Exploration

```
df.describe()
```

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Item_Outlet_Sales
count	7060.000000	8523.000000	8523.000000	8523.000000	8523.000000
mean	12.857645	0.066132	140.992782	1997.831867	2181.288914
std	4.643456	0.051598	62.275067	8.371760	1706.499616
min	4.555000	0.000000	31.290000	1985.000000	33.290000
25%	8.773750	0.026989	93.826500	1987.000000	834.247400
50%	12.600000	0.053931	143.012800	1999.000000	1794.331000
75%	16.850000	0.094585	185.643700	2004.000000	3101.296400
max	21.350000	0.328391	266.888400	2009.000000	13086.964800

Hasil Pengamatan

Item_Visibility: Nilai minimumnya bernilai 0.00, diperlukan eksplorasi lebih dalam untuk kolom ini.

nilai minimal pada item_visibility bernilai 0,0 perlu diperiksa apakah dibutuhkan imputasi atau tidak karena barang yg tidak terlihat di rak masih bisa terjual

Data Exploration

Eksplorasi lebih dalam kolom Item_Visibility

```
missing_rak = df[df['Item_Visibility'] == 0]
missing_rak
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type
3	FDX07	19.200	Regular	0.0	Fruits and Vegetables	182.0950	OUT010	1998	NaN	T
4	NCD19	8.930	Low Fat	0.0	Household	53.8614	OUT013	1987	High	T
5	FDP36	10.395	Regular	0.0	Baking Goods	51.4008	OUT018	2009	Medium	T
10	FDY07	11.800	Low Fat	0.0	Fruits and Vegetables	45.5402	OUT049	1999	Medium	T
32	FDP33	18.700	Low Fat	0.0	Snack Foods	256.6672	OUT018	2009	Medium	T
...	...	...	...	...	...	...	...	...	...	...
8480	FDQ58	NaN	Low Fat	0.0	Snack Foods	154.5340	OUT019	1985	Small	T
8484	DRJ49	6.865	Low Fat	0.0	Soft Drinks	129.9652	OUT013	1987	High	T
8486	FDR20	20.000	Regular	0.0	Fruits and Vegetables	46.4744	OUT010	1998	NaN	T
8494	NCI54	15.200	Low Fat	0.0	Household	110.4912	OUT017	2007	NaN	T
8500	NCQ42	20.350	Low Fat	0.0	Household	125.1678	OUT017	2007	NaN	T

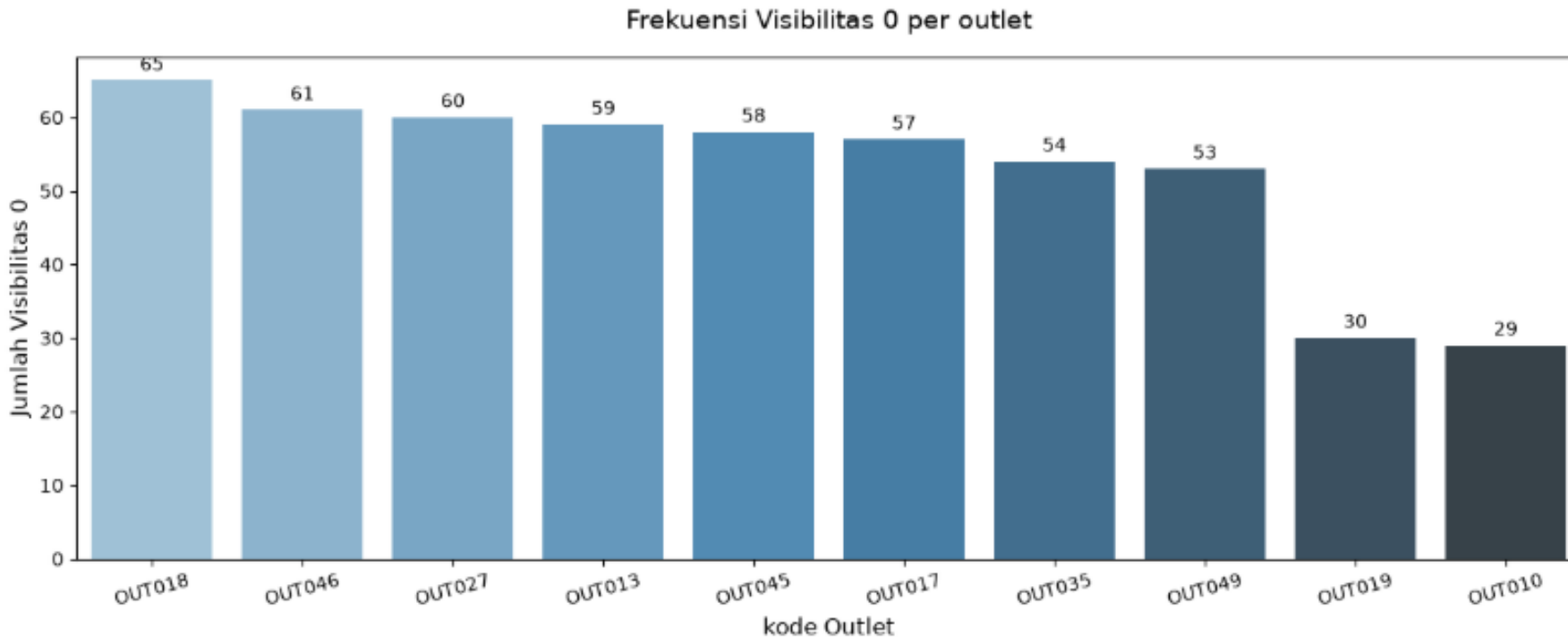
526 rows × 12 columns

Hasil Pengamatan

Kolom_Visibility yang bernilai 0 berjumlah 526 baris dan memiliki total pendapatan dari penjualan produk tertentu. Dalam dunia retail hal ini sangat mustahil karena produk yang tidak dipajang tidak mungkin terjual oleh konsumen. dibutuhkan eksplorasi lebih lanjut untuk mengatasi kolom Item_Visibility bernilai 0

Data Exploration

Melihat persebaran data visibilitas 0



Hasil Pengamatan

Berdasarkan pada grafik diatas produk dengan visibilitas 0 dan memiliki pendapatan penjualan tersebar diseluruh outlet, dapat disimpulkan sebagai kesalahan input. Produk dengan visibilitas 0 akan dikosongkan (bukan angka 0) yang selanjutnya akan diganti dengan nilai yang sesuai untuk membuat data konsisten

Data Exploration

Imputasi pada kolom Item_Visibility

```
# membuat data salinan sementara sebelum cleaning  
df_semi=df.copy()
```

```
df_semi['Item_Visibility'] = df_semi['Item_Visibility'].replace(0, np.nan)  
# cek nilai yang kosong  
df_semi['Item_Visibility'].isna().sum()
```

```
np.int64(526)
```

```
df['Item_Visibility'].isna().sum()
```

```
np.int64(0)
```

Hasil Pengamatan

pada salinan dataframe kolom Item_visibility memiliki missing value sebanyak 526 baris. akan diperlukan perhitungan persentase untuk keputusan selanjutnya

Data Exploration

Cek Data Hilang

```
#cek persentase missing value
df_miss = pd.DataFrame({
    'data_hilang' : df_clean.isna().sum(),
    'Percentage' : (df_clean.isna().mean()*100).round(2)
})
df_miss = df_miss[df_miss['data_hilang'] > 0]
df_miss
```

	data_hilang	Percentage
Item_Weight	1463	17.17
Item_Visibility	526	6.17
Outlet_Size	2410	28.28

Hasil Pengamatan

kolom Item_Weight memiliki persentase 17,17% diperlukan pengujian lebih lanjut
kolom Item_Visibility memiliki persentase 6,17% diperlukan pengujian lebih lanjut
kolom Outlet_Size memiliki persentase 28.28% diperlukan pengujian lebih lanjut

Data Exploration

Cek Data Hilang & duplikasi data

```
#cek persentase missing value
df_miss = pd.DataFrame({
    'data_hilang' : df_clean.isna().sum(),
    'Percentage' : (df_clean.isna().mean()*100).round(2)
})
df_miss = df_miss[df_miss['data_hilang'] > 0]
df_miss
```

	data_hilang	Percentage
Item_Weight	1463	17.17
Item_Visibility	526	6.17
Outlet_Size	2410	28.28

```
#cek duplikat setiap baris
duplikat_jml = df_clean.duplicated().sum()
duplikat_persen = (duplikat_jml / len(df)) * 100
print(f"Jumlah baris duplikat: {duplikat_jml} ({duplikat_persen:.2f}%)"
Jumlah baris duplikat: 0 (0.00%)
```

Hasil Pengamatan

kolom Item_Weight memiliki persentase 17,17% diperlukan pengujian lebih lanjut

kolom Item_Visibility memiliki persentase 6,17% diperlukan pengujian lebih lanjut

kolom Outlet_Size memiliki persentase 28.28% diperlukan pengujian lebih lanjut

Tidak terdapat data yang duplikat

Data Exploration

Cek konsistensi data

	column	unique_values	n_unique
0	Item_Identifier	[FDA15, DRC01, FDN15, FDX07, NCD19, FDP36, FDO10, FDP10, FDH17, FDU28, FDY07, FDA03, FDX32, FDS46, FDF32, FDP49, NCB42, DRI11, FDU02, FDN22, FDW12, NCB30, FDC37, FDR28, NCD06, FDV10, DRI59, FDE51, FDC14, FDV38, NCS17, FDP33, FDO23, DRH01, NCX29, FDV20, DRZ11, FDX10, FDB34, FDK43, FDA46, FDC02, FDL50, FDM39, NCP05, FDV49, FDL12, FDS02, NCL17, FDM40, FDR13, FDA43, NCP18, FDK21, NCX54, DRK35, FDY21, FDI26, FDM20, FDV27, FDF09, FDY40, FDY45, FDC46, FDH19, FDZ03, DRH37, NCI17, FDJ58, FDH35, FDG02, NCZ18, FDC29, FDQ10, FDN48, FDL04, FDV25, FDD58, FDN04, FDV45, NCL18, FDR12, FDG20, FDZ55, FDQ49, FDN33, FDN27, FDW20, DRG27, DRI25, FDA44, NCR17, FDU04, FDF41, FDB56, FDT28, FDD10, FDW57, DRB48, FDP09, ...]	1559
1	Item_Fat_Content	[Low Fat, Regular, low fat, LF, reg]	5
2	Item_Type	[Dairy, Soft Drinks, Meat, Fruits and Vegetables, Household, Baking Goods, Snack Foods, Frozen Foods, Breakfast, Health and Hygiene, Hard Drinks, Canned, Breads, Starchy Foods, Others, Seafood]	16
3	Outlet_Identifier	[OUT049, OUT018, OUT010, OUT013, OUT027, OUT045, OUT017, OUT046, OUT035, OUT019]	10
4	Outlet_Size	[Medium, High, Small]	3
5	Outlet_Location_Type	[Tier 1, Tier 3, Tier 2]	3
6	Outlet_Type	[Supermarket Type1, Supermarket Type2, Grocery Store, Supermarket Type3]	4

```
# Export hasil nilai unik per kolom untuk dianalisa oleh AI memeriksa nilai yang tidak konsisten
df_ceksdf.to_csv('nilai unique.csv', index=False)
```

Hasil Pengamatan

terdapat inkonsistensi nilai pada kolom Item_Fat_Content dan nilai pada kolom lain sudah konsisten (tidak redundant)

Data Cleaning

Cek Inkonsistensi Data

```
# cek inkonsistensi nilai pada kolom Item_Fat_Content  
df['Item_Fat_Content'].unique()
```

```
<ArrowStringArray>  
['Low Fat', 'Regular', 'low fat', 'LF', 'reg']  
Length: 5, dtype: str
```

```
# aksi untuk menyeragamkan inkonsistensi data Item_Fat_Content  
df_clean['Item_Fat_Content'] = df['Item_Fat_Content'].replace({  
    'low fat': 'Low Fat',  
    'LF': 'Low Fat',  
    'reg': 'Regular'  
})  
df_clean['Item_Fat_Content'].unique()
```

```
<ArrowStringArray>  
['Low Fat', 'Regular']  
Length: 2, dtype: str
```

Hasil Cleaning

dilakukan penyeragaman nilai Low Fat, low fat, LF menjadi Low Fat begitu juga untuk Regular, reg menjadi Regular

Data Cleaning

Penanganan Missing Value pada kolom Item_Weight

```
# Cek konsistensi berat pada kolom Item_Weight
weight_check = df_clean.groupby('Item_Identifier')['Item_Weight'].nunique()
print(f"Jumlah produk dengan variasi berat > 1: {(weight_check > 1).sum()}")

#menampilkan hasil data dengan Item_Identifier menghasilkan berat yang sama
sample_id = df_clean[df_clean['Item_Weight'].isna()]['Item_Identifier'].iloc[2]
print(f"\nContoh perbandingan untuk Item_Identifier '{sample_id}':")

df_clean[df_clean['Item_Identifier'] == sample_id][['Item_Identifier', 'Item_Weight', 'Outlet_Identifier']]
```

Jumlah produk dengan variasi berat > 1: 0

Contoh perbandingan untuk Item_Identifier 'FDW12':

	Item_Identifier	Item_Weight	Outlet_Identifier
21	FDW12	NaN	OUT027
1788	FDW12	8.315	OUT010
2973	FDW12	8.315	OUT049

Hasil Pengamatan

berdasarkan pengamatan dengan produk yang sama menggunakan kolom Item_Identifier nilai berat produk sama semuanya maka akan dilakukan Imputasi pada nilai Item_Weight yang kosong dengan menyesuaikan dengan Item_Identifier

Data Cleaning

Penanganan Missing Value pada kolom Item_Weight

```
# Aksi imputasi Item_Weight menggunakan nilai rata-rata dari Item_Identifier yang sama
df_clean['Item_Weight'] = df_clean.groupby('Item_Identifier')['Item_Weight'].transform(lambda x: x.fillna(x.mean()))

# Cek kembali jumlah missing value pada kolom Item_Weight setelah diimputasi
print("Sisa missing value pada Item_Weight:", df_clean['Item_Weight'].isna().sum())
```

Sisa missing value pada Item_Weight: 4

```
#Imputasi 4 nilai yang kosong dengan rata-rata kolom Item_Weight
df_clean['Item_Weight'] = df_clean['Item_Weight'].fillna(df_clean['Item_Weight'].mean())

# Cek kembali jumlah missing value pada kolom Item_Weight setelah diimputasi
print("Sisa missing value pada Item_Weight:", df_clean['Item_Weight'].isna().sum())
```

Sisa missing value pada Item_Weight: 0

Hasil Cleaning

Imputasi Tahap 1 (Berdasarkan ID Produk): Sebanyak 1.459 missing value pada Item_Weight berhasil diisi menggunakan rata-rata berat dari Item_Identifier yang sama, karena terbukti setiap produk memiliki berat yang konsisten (berat = 0).

Imputasi Tahap 2 (Penanganan Sisa 4 Data): Sisa 4 data yang masih kosong (karena produk tersebut berdiri sendiri dan tidak memiliki pembanding ID lain) diimputasi menggunakan nilai rata-rata keseluruhan (overall mean) untuk mempertahankan data Item_Outlet_Sales.

Validasi Akhir: Sisa missing value pada kolom Item_Weight berhasil menjadi 0 baris (100% bersih).

Data Cleaning

Penanganan Missing Value pada kolom Outlet_Size

```
#Uji hubungan antar 2 kolom
pd.crosstab(df_clean['Outlet_Type'], df_clean['Outlet_Size'], dropna=False)
```

Outlet_Size	High	Medium	Small	NaN
Outlet_Type				
Grocery Store	0	0	528	555
Supermarket Type1	932	930	1860	1855
Supermarket Type2	0	928	0	0
Supermarket Type3	0	935	0	0

```
# uji lebih lanjut untuk nilai yang hilang dengan tipe Supermarket Type1 dengan kolom Outlet_Location_Type
sup1_df = df_clean[df_clean['Outlet_Type'] == 'Supermarket Type1']

# Crosstab antara Outlet_Location_Type dan Outlet_Size= Supermarket Type1
pd.crosstab(sup1_df['Outlet_Location_Type'], sup1_df['Outlet_Size'], dropna=False)
```

Outlet_Size	High	Medium	Small	NaN
Outlet_Location_Type				
Tier 1	0	930	930	0
Tier 2	0	0	930	1855
Tier 3	932	0	0	0

Hasil Pengamatan

karena tipe sebuah outlet mempengaruhi sebuah ukuran outlet maka akan diuji hubungan antar kolom Outlet_Size dengan kolom Outlet_Type

terdapat data yang hilang pada data bertipe Supermarket Type 1 dan Grocery. Data hilang yang bertipe Grocery terkonsentrasi dalam ukuran small, data Supermarket Type1 tersebar di setiap ukuran dibutuhkan uji lebih lanjut dengan kolom lain yang berhubungan dengan Outlet_Type yang bernilai Supermarket Type1 dengan Outlet_location Size

terlihat dari tabel nilai kosong berada pada Tier 2 dan jumlah High dan Medium sudah sama dengan tabel yang dilakukan uji pertama kali sehingga nilai kosong yang bertipe Supermarket Type1 di imputasi dengan ukuran small, jadi semua data Outlet_Size yang kosong akan di isi dengan ukuran small

Data Cleaning

Penanganan Missing Value pada kolom Outlet_Size

```
# Imputasi nilai yang kosong pada Outlet_Size
df_clean['Outlet_Size'] = df_clean['Outlet_Size'].fillna('Small')
# Cek kembali jumlah missing value pada kolom Outlet_Size setelah diimputasi
print("Sisa missing value pada Outlet_Size:", df_clean['Outlet_Size'].isna().sum())
```

Sisa missing value pada Outlet_Size: 0

```
#cek apakah data yang di imputasi terisi dengan benar
pd.crosstab(df_clean['Outlet_Type'], df_clean['Outlet_Size'], dropna=False)
```

	Outlet_Size	High	Medium	Small
Outlet_Type				
Grocery Store		0	0	1083
Supermarket Type1		932	930	3715
Supermarket Type2		0	928	0
Supermarket Type3		0	935	0

Hasil Cleaning

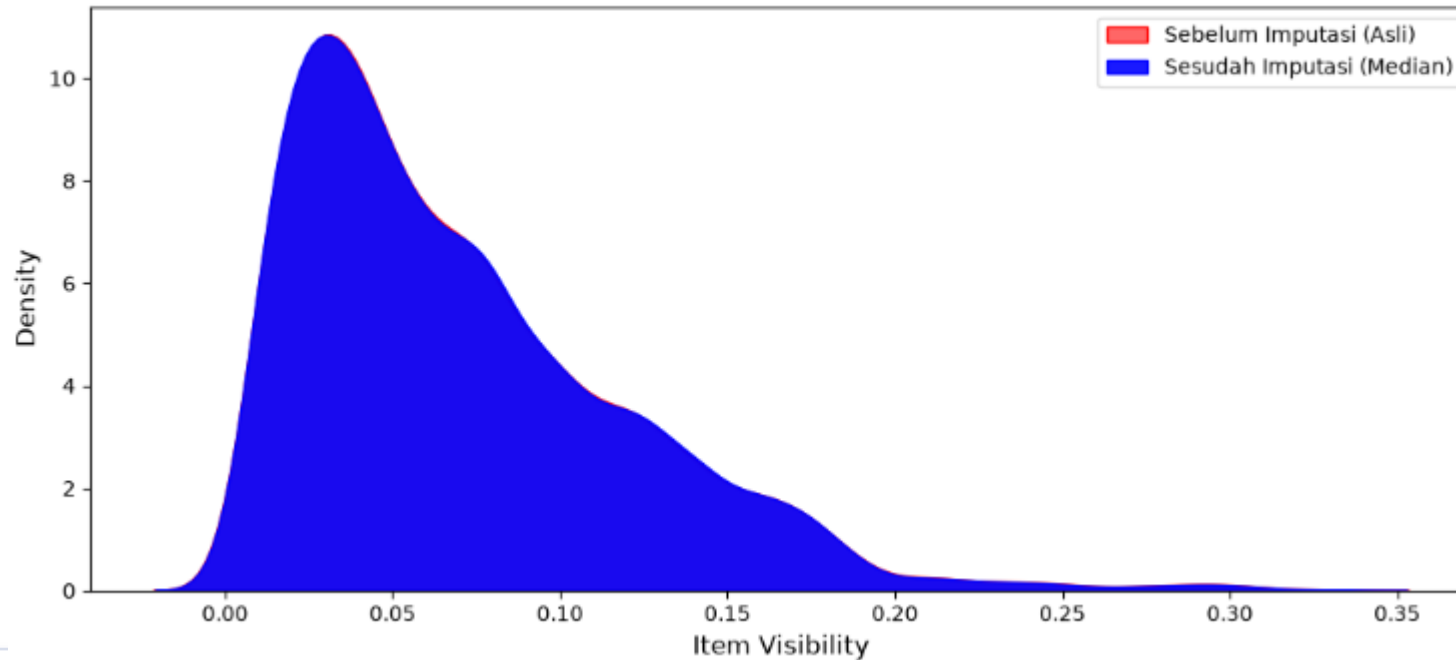
karena tipe sebuah outlet mempengaruhi sebuah ukuran outlet maka akan diuji hubungan antar kolom Outlet_Size dengan kolom Outlet_Type

Data Cleaning

Penanganan Missing Value pada kolom Item_Visibility

```
df_clean['Item_Visibility'] = df_clean['Item_Visibility'].fillna(  
    df_clean.groupby('Item_Identifier')['Item_Visibility'].transform('median')  
)  
  
# Cek apakah masih ada missing  
print("Sisa missing value pada Item_Visibility:", df_clean['Item_Visibility'].isna().sum())
```

Sebelum vs Sesudah Imputasi



Hasil Cleaning

nilai yang kosong akan diisi dengan nilai tengah dari kolom Item_Visibility berdasarkan produknya
hasil dari visualisasi hampir menyerupai dengan data aslinya

Data Cleaning

5. Mengidentifikasi Outlier dan Menangani Outlier

```
'''cek Outlier dengan IQR
'''

# Cek outlier untuk kolom Item_Outlet_Sales
Q1 = df_clean['Item_Outlet_Sales'].quantile(0.25)
Q3 = df_clean['Item_Outlet_Sales'].quantile(0.75)

IQR = Q3 - Q1

batas_bawah = Q1 - (1.5 * IQR)
batas_atas = Q3 + (1.5 * IQR)

print(f"Batas Bawah: {batas_bawah}")
print(f"Batas Atas: {batas_atas}")

outlier_bawah = df_clean[df_clean['Item_Outlet_Sales'] < batas_bawah]
outlier_atas = df_clean[df_clean['Item_Outlet_Sales'] > batas_atas]
print(f"Jumlah baris di bawah batas normal: {len(outlier_bawah)} baris")
print(f"Jumlah baris data dengan penjualan di atas batas normal (outlier): {len(outlier_atas)} baris")

Batas Bawah: -2566.3261
Batas Atas: 6501.8699
Jumlah baris di bawah batas normal: 0 baris
Jumlah baris data dengan penjualan di atas batas normal (outlier): 186 baris
```

```
# Cek outlier untuk kolom Item_Visibility
Q1_vis = df_clean['Item_Visibility'].quantile(0.25)
Q3_vis = df_clean['Item_Visibility'].quantile(0.75)
IQR_vis = Q3_vis - Q1_vis
batas_atas_vis = Q3_vis + (1.5 * IQR_vis)

print(f"Batas Atas Item_Visibility: {batas_atas_vis}")
outlier_visibilitas_tinggi = df_clean[df_clean['Item_Visibility'] > batas_atas_vis]
print(f"Jumlah baris data dengan Item_Visibility di atas batas normal: {len(outlier_visibilitas_tinggi)} baris")

Batas Atas Item_Visibility: 0.19157102725000003
Jumlah baris data dengan Item_Visibility di atas batas normal: 154 baris

.. .. -
```

Hasil Cleaning

Outlier Penjualan (Item_Outlet_Sales): Teridentifikasi 186 baris data di atas batas normal IQR (> 6.501,87).

Outlier Visibilitas (Item_Visibility): Teridentifikasi 154 baris data di atas batas normal IQR (> 0,1916).

Keputusan Penanganan: Seluruh outlier dipertahankan (tidak dihapus) karena mewakili kondisi bisnis nyata (high-performing products/outlets), bukan disebabkan oleh kesalahan input data (data entry error).

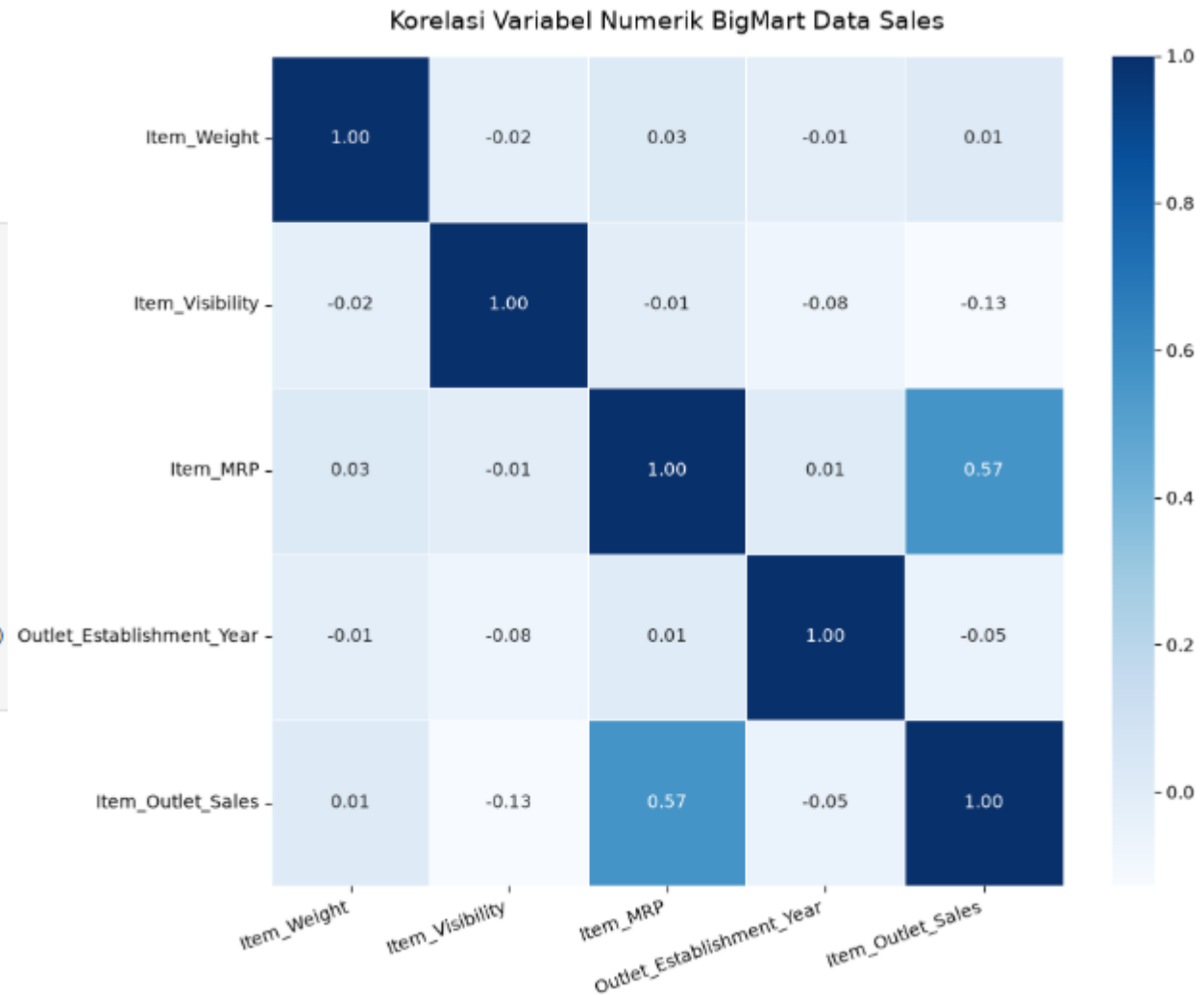
Data Manipulation

Penentuan analisa

```
plt.figure(figsize=(10, 8))

df_numerik = df_clean.select_dtypes(include=['number']).corr()
sns.heatmap(
    df_numerik,
    annot=True,
    cmap='Blues',
    fmt='.2f',
    linewidths=0.5,
    cbar=True
)
plt.xticks(rotation=20, ha='right')
plt.yticks(rotation=0)

plt.title('Korelasi Variabel Numerik BigMart Data Sales', fontsize=14, pad=15)
plt.tight_layout()
plt.show()
```



Hasil Pengamatan

kolom Item_MRP dengan kolom Item_Outlet_Sales memiliki korelasi paling kuat sebesar 0.57, maka arah analisa akan menuju kepada dua kolom ini

Data Manipulation

filter kolom yang dibutuhkan untuk analisa

```
#filter kolom yang dibutuhkan untuk analisa
kolom_cek = ['Item_Type', 'Item_MRP', 'Outlet_Type', 'Outlet_Location_Type', 'Outlet_Size', 'Item_Outlet_Sales']

df_analysis = df_clean[kolom_cek].copy()

''' Membulatkan nilai Item_Outlet_Sales menjadi 2 angka di belakang koma
    Membulatkan nilai Item_MRP menjadi 2 angka di belakang koma
    ...
df_analysis['Item_Outlet_Sales'] = df_analysis['Item_Outlet_Sales'].round(2)
df_analysis['Item_MRP'] = df_analysis['Item_MRP'].round(2)

df_analysis.head()
```

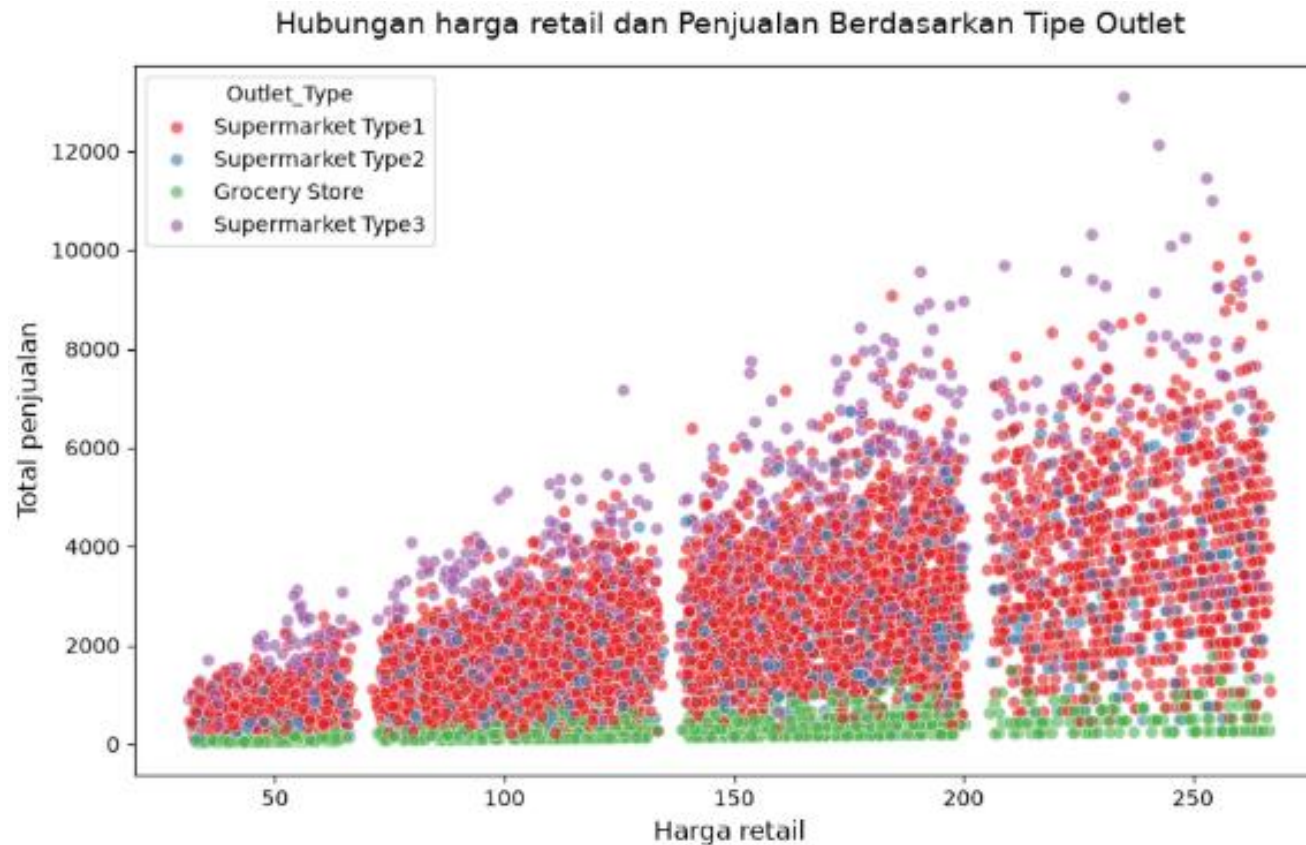
	Item_Type	Item_MRP	Outlet_Type	Outlet_Location_Type	Outlet_Size	Item_Outlet_Sales
0	Dairy	249.81	Supermarket Type1	Tier 1	Medium	3735.14
1	Soft Drinks	48.27	Supermarket Type2	Tier 3	Medium	443.42
2	Meat	141.62	Supermarket Type1	Tier 1	Medium	2097.27
3	Fruits and Vegetables	182.10	Grocery Store	Tier 3	Small	732.38
4	Household	53.86	Supermarket Type1	Tier 3	High	994.71

Hasil Pengamatan

kolom Item_MRP dengan kolom Item_Outlet_Sales memiliki korelasi paling kuat sebesar 0.57, maka arah analisa akan menuju kepada dua kolom ini

Data Manipulation

Visualisasi harga retail dan Penjualan Berdasarkan Tipe Outlet

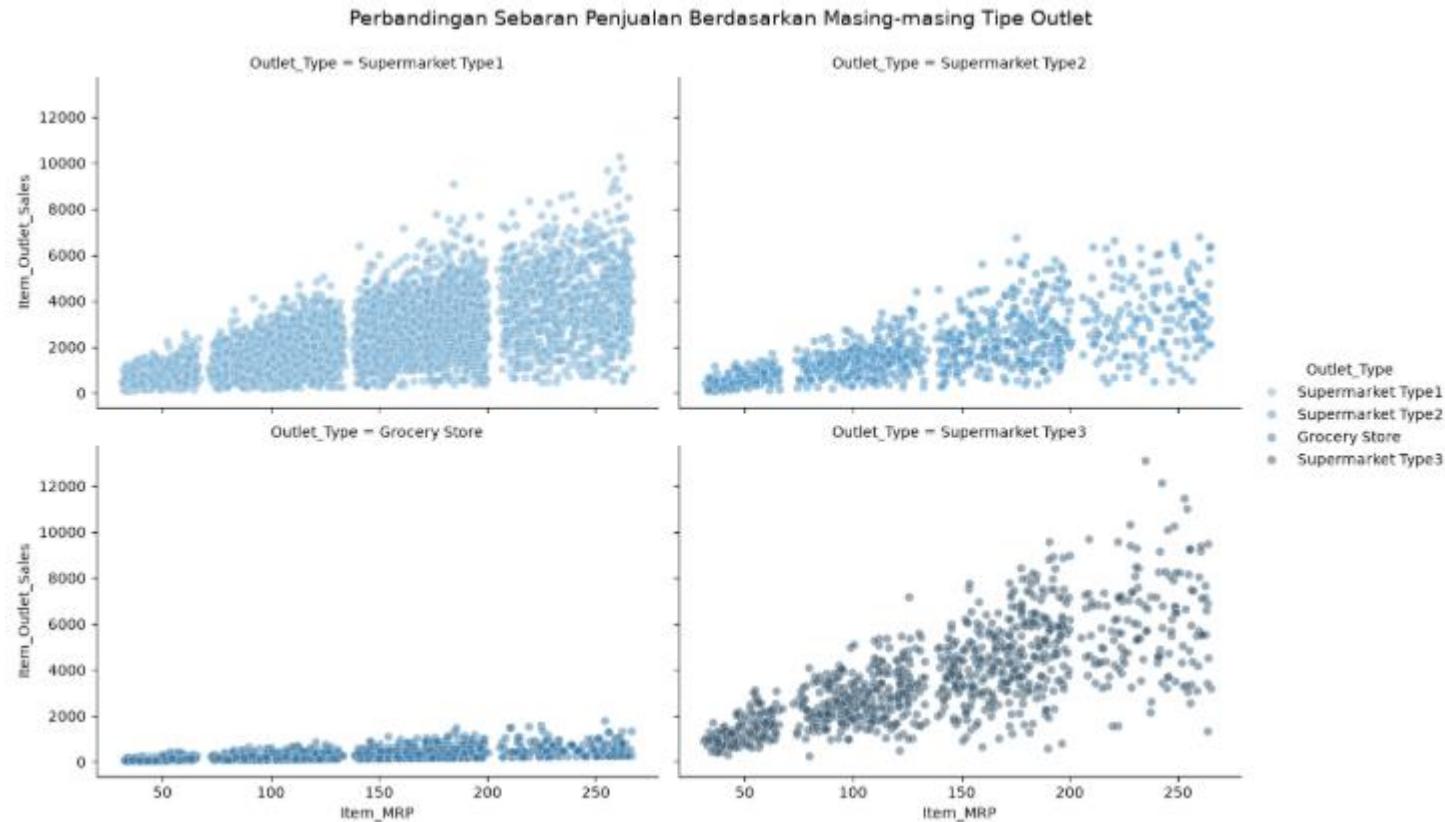


Hasil Pengamatan

Dari grafik ini dapat dilihat bahwa harga retail memang menentukan batas potensi penjualan. Semakin mahal barangnya, batas atau penjualannya makin tinggi. Tetapi, berdasarkan titik-titik hijau di bagian bawah; meskipun sebuah produk berharga mahal dijual di Grocery Store, omzetnya tetap tertahan di bawah. Sebaliknya, Supermarket Type 3 (warna ungu) secara konsisten mampu mendorong penjualan ke level tertinggi di setiap kategori harga. Artinya, pemilihan tipe outlet jauh lebih krusial dalam menentukan volume omzet akhir.

Data Manipulation

Visualisasi Sebaran Penjualan Berdasarkan Masing-masing Tipe Outlet

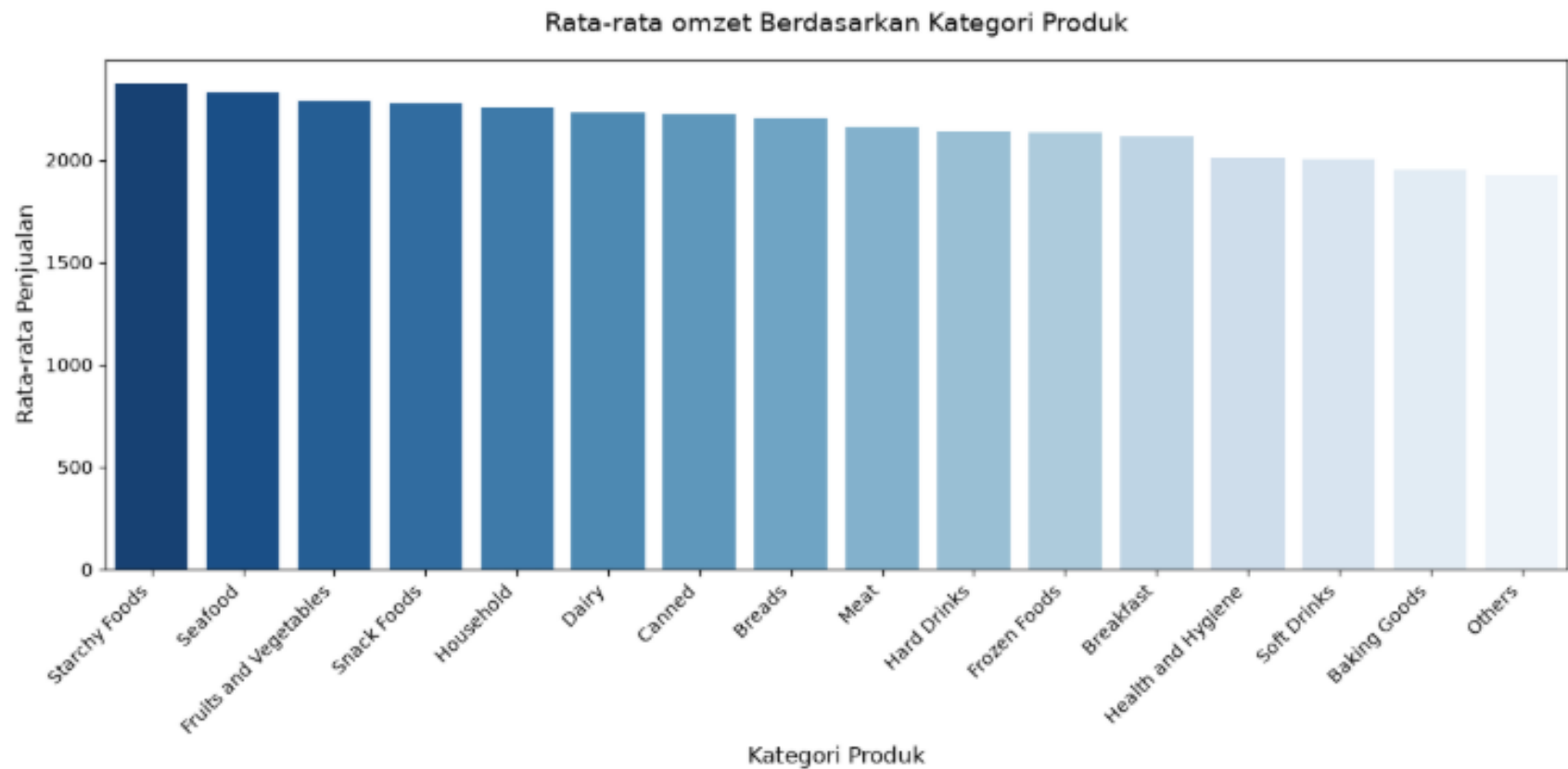


Hasil Pengamatan

Pemisahan panel ini membuktikan secara mutlak bahwa tipe outlet adalah pembeda performa yang sangat ekstrem. Jika sebelumnya pada grafik gabungan titik-titiknya tampak menumpuk, pemecahan ini memperjelas mengapa Supermarket Type 3 bisa menghasilkan penjualan belasan ribu, sementara Grocery Store hanya mampu mencetak angka di bawah 2.000.

Data Manipulation

Visualisasi Rata-rata omzet Berdasarkan Kategori Produk



Hasil Pengamatan

Secara keseluruhan, rata-rata omzet tiap kategori produk cenderung stabil dan membentuk kurva landai dengan selisih yang sangat tipis (~1.900 hingga 2.350). Kategori Starchy Foods, Seafood, Fruits and Vegetables, serta Snack Foods memimpin di jajaran teratas. Temuan ini mengindikasikan bahwa konsumen BigMart berbelanja secara merata di berbagai lini produk, sehingga strategi ritel harus fokus pada ketersediaan stok yang konsisten di semua kategori, bukan hanya mengandalkan produk tertentu saja

Hasil Analisa



Korelasi Positif antara Harga Produk dan Potensi Pendapatan Penjualan

Analisis awal menunjukkan adanya hubungan linier positif antara harga eceran produk (Item_MRP) dengan total pendapatan penjualan. Semakin tinggi harga suatu produk, semakin besar pula potensi batas maksimal omzet yang dapat dihasilkan oleh sistem penjualan.



Tipe Outlet Merupakan Faktor Penentu Utama Variasi Omzet

Pengujian rata-rata omzet berdasarkan kategori produk memperlihatkan selisih nilai yang rapat di kisaran angka 1.900 hingga 2.400. Hal ini membuktikan bahwa jenis barang bukanlah faktor pemicu utama yang menciptakan perbedaan omzet secara drastis di antara toko.



Kategori Produk (Item_Type) Memiliki Performa Penjualan yang Relatif Stabil dan Merata

Pengujian rata-rata omzet berdasarkan kategori produk memperlihatkan selisih nilai yang rapat di kisaran angka 1.900 hingga 2.400. Hal ini membuktikan bahwa jenis barang bukanlah faktor pemicu utama yang menciptakan perbedaan omzet secara drastis di antara toko.

Rekomendasi Bisnis



Ekspansi dan Modernisasi Tipe Outlet

Prioritas pengembangan bisnis dapat diarahkan pada ekspansi format Supermarket (Type 1 & Type 3), mengingat data membuktikan tipe gerai ini memiliki leverage paling efektif dalam mendorong volume omzet secara signifikan.



Promosi Berbasis Kategori produk

Manajemen dapat mempertimbangkan skema promosi terpadu (bundling) pada kategori produk unggulan, sebagai opsi strategis untuk meningkatkan nilai rata-rata keranjang belanja per transaksi.